Build My First RAG — Step-by-Step Guide

Nov 4, 2026 · @Meera Ramesh

What you're building

You're building a study assistant that answers questions from your own `rag-docs` folder (68 Markdown files, about 1.37 million words) and points to the file and section each answer came from. Everything runs free on your Mac except the final answer step, which calls Claude.

The idea in one line: RAG = *Retrieval-Augmented Generation*. Before the AI answers, your code **retrieves** the few most relevant passages from your notes and **augments** the prompt with them, so the AI **generates** an answer grounded in your material instead of guessing.

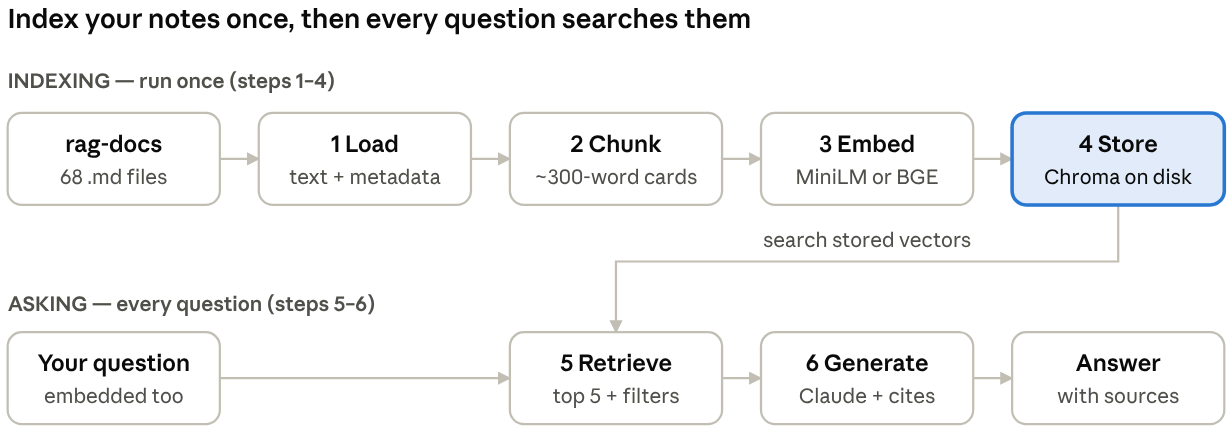
Explain it like I'm 10: It's an open-book test. The AI doesn't have to memorize your notes. It just needs a really fast librarian who finds the right 5 pages and puts them on the desk before it writes the answer.

Real-world example: Company help bots work exactly like this. The bot searches the help-center articles first, then writes a reply using only what it found. Your version searches your SQL, Java and system design notes instead.

Your free local stack:

Piece	Tool	Cost	Why this one
Language	Python 3.10+	Free	Every RAG library supports it first
Embedding model	`sentence-transformers` with `all-MiniLM-L6-v2` or `BAAI/bge-small-en-v1.5`	Free, runs on your laptop	Small, fast, good enough for study notes
Vector database	Chroma	Free, runs on your laptop	No account or server; saves to a folder
Answer writer (LLM)	Claude via the Anthropic API	Pennies per question	Reads the retrieved chunks and writes the answer

The build has two halves. **Indexing** happens once: steps 1 to 4 load, chunk, embed and store your notes. **Asking** happens every time you ask a question: steps 5 and 6 retrieve and generate. Step 7 tests whether it works.



RAG pipeline · indexing once, asking every time

The Chroma store is the hand-off point: indexing fills it once, and every question searches it.

Step 0 — Set up your workspace

Goal: a clean project folder with its own Python environment, the four libraries, and your API key. This should take about 15 minutes.

Explain it like I'm 10: Before baking, you clear the counter and put out the bowls. A virtual environment is your own counter, so this project's ingredients don't get mixed up with other projects.

1. **Make a project folder next to your docs**, for example `lstemp/my-first-rag/`. Keep the code outside `rag-docs` so the docs folder stays pure data.
2. **Create a virtual environment** inside it with Python's built-in `venv` module, then activate it. Every package you install now lives only in this project.
3. **Install the libraries:**
   - `chromadb` is the vector database.
   - `sentence-transformers` is the free embedding model. It pulls in PyTorch, so the first install is large.
   - `anthropic` is the client for calling Claude.
   - `python-frontmatter` (or plain `PyYAML`) reads the header at the top of each file.

4. **Get an Anthropic API key** from the Claude Console and save it as an environment variable named `ANTHROPIC_API_KEY`. Never paste the key into your code or commit it to GitHub.
5. **Plan your files.** A simple layout is one script per half: `index.py` (steps 1–4, run once) and `ask.py` (steps 5–6, run per question). Chroma will create a `chroma_db/` folder when you first save.

What you're feeding it (from `rag-docs/README.md`):

`source_type`	What it is	Approx. words
`book`	Text PDFs: Spring Boot notes, topmate Java and System Design, tutorials	715k
`my-notes`	Your own Markdown notes across 10 topics	465k
`interview-questions`	Your question files, Goldman Sachs PDF	126k
`ocr`	12 scanned PDFs read by OCR (noisy text)	63k

Real-world tip: start with one topic folder (for example `databases/`) while you build. Indexing 5 files takes seconds, so you can rerun often. Point it at the full folder only once everything works.

Step 1 — Load the documents

Goal: turn the 68 files into a Python list where each item holds the file's **text** and its **metadata** (topic, source type, original file, title).

Explain it like I'm 10: You're taking every notebook off the shelf and reading the label on its cover before you open it. The label tells you which subject it belongs to.

1. **Find every file.** Walk `rag-docs/` recursively for `*.md` files and skip `README.md`, which describes the folder and isn't study content.
2. **Split the header from the body.** Each file starts with a frontmatter block between two `---` lines, like this:

```
title: SQL-Complete-Notes
topic: databases
source_type: my-notes
source_file: STUDYNOTEs/SQL-Complete-Notes.md
```

A frontmatter library splits it into a dictionary (the metadata) and a string (the body).

3. **Keep both together.** Store each document as `{text, metadata}`. Every chunk you make in Step 2 copies this metadata, so a chunk always knows where it came from.
4. **Sanity-check the count.** Print how many files loaded per topic. You should see 68 files across 10 topics; a lower number means a path or glob is wrong.

Why metadata matters (real-world example): when your RAG answers "What is a LEFT JOIN?", you want it to say *databases › sql-complete-notes* and not a page from the Spring Boot book. Metadata is also what lets you filter searches later ("only my notes", "only system design").

Watch out: the PDF-based files contain lines like `<!-- page 12 -->`. Don't strip them at this step. Step 2 uses them to split books by page and to cite page numbers.

Step 2 — Chunk the documents

Goal: cut each document into pieces of about **150–400 words**, each holding one idea, using a different cutting rule for each `source_type`. This is the step that most decides how good your RAG is.

Explain it like I'm 10: You're cutting a giant textbook into index cards. A card with one word is useless, and a card with a whole chapter is too much to scan. You want one idea per card, and you write the chapter name at the top of each card so you know where it came from.

Why not one chunk per file? The search compares your question to each chunk as a whole. A 60,000-word Java file would match everything a little and nothing well. Small embedding models also only "read" the first ~250–500 words of a chunk and ignore the rest.

The cutting rule for each kind of file:

source_type	Cut on	Example in your files
my-notes	Markdown headings (##, ###)	SQL-Complete-Notes has 260 headings, so roughly one concept per chunk
interview-questions	Each question line	SQL, Java and Python files mark questions as bold lines like **Q1. What is SQL?** or **1. What is Java?**; the flashcard file uses ### Q: headings
book	The <!-- page N --> markers	A 1,336-page Spring Boot PDF becomes page-sized chunks
ocr	The <!-- page N --> markers	Same as books, but expect messy text

Step by step:

1. **Split on structure first.** Use the rule from the table to get natural sections. LangChain's MarkdownHeaderTextSplitter (or a simple regex on lines starting with #) does heading splits for you.
2. **Merge tiny pieces.** A heading with one sentence under it (for example, a table of contents line) gets joined to the next section. Aim for at least ~50 words.
3. **Cut long pieces with overlap.** Anything over ~400 words gets split into windows of about 300 words, each sharing ~50 words with the next. The overlap means a sentence cut at a boundary still appears whole in one chunk.
4. **Add the heading path to each chunk.** Store it as metadata, for example section: SQL Complete Notes > Joins > LEFT JOIN. Also prefix it to the chunk text itself, so a chunk that just says "use it when..." still carries its topic into the embedding.
5. **Give each chunk a stable ID**, for example databases/sql-complete-notes#joins-left-join-2. Stable IDs let you re-index without creating duplicates.
6. **Check the result.** Print the number of chunks per topic and a few random chunks. With ~1.37M words and ~300-word chunks, expect a few thousand chunks in total.

Real-world example: in your interview-questions files, a chunk should be exactly one question and its answer. Then "difference between WHERE and HAVING" retrieves that one Q&A, not half of it glued to the next question about GROUP BY.

Watch out: don't split inside a code block. A chunk that ends halfway through a SQL query is confusing for both the search and Claude. When you cut, check whether you're between a pair of ``` fences.

Step 3 — Embed the chunks

Goal: turn every chunk into an **embedding**, a list of a few hundred numbers that captures what the chunk *means*, using a free model that runs on your laptop.

Explain it like I'm 10: Imagine a giant map where every index card gets a pin. Cards about similar ideas get pinned close together: "LEFT JOIN" sits near "OUTER JOIN", far from "React useState". An embedding is just the map coordinates for one card. Later your question gets a pin too, and you grab the cards nearest to it.

Why meaning beats keywords (real-world example): you ask "how do I find customers who never ordered?" Your SQL notes never use the word *never*, but they have a section on `LEFT JOIN ... WHERE ... IS NULL`. A keyword search misses it; an embedding search finds it because the two mean the same thing.

Pick a model:

Model	Numbers per chunk	Reads up to	When to choose it
`all-MiniLM-L6-v2`	384	~256 tokens (~190 words)	Fastest. Good for a first version
`BAAI/bge-small-en-v1.5`	384	512 tokens (~380 words)	Usually more accurate, and fits your ~300-word chunks fully

1. **Load the model once** with `sentence-transformers`. The first run downloads it (under 150 MB); after that it works offline.
2. **Match chunk size to the model.** Anything past the model's limit is silently cut off. This is why Step 2 caps chunks around 300 words.
3. **Embed in batches** (for example, 64 chunks at a time) rather than one by one. For a few thousand chunks this takes minutes on a laptop CPU, not hours.
4. **Use the same model for questions.** Your questions in Step 5 must be embedded with the exact same model. Vectors from two different models are like map coordinates from two different maps.

5. **Follow the model's instructions.** BGE models recommend adding a short instruction prefix to *questions* (not chunks) for search. Check the model card on Hugging Face.

Shortcut: Chroma can do this step for you. If you give a collection an embedding function (it has one for `sentence-transformers`), it embeds chunks when you add them and questions when you search. Doing it by hand once first helps you understand what's happening.

Step 4 — Store the chunks in Chroma

Goal: save every chunk's **ID, text, embedding and metadata** in a Chroma collection on disk, so you index once and ask questions forever after.

Explain it like I'm 10: A vector database is a filing cabinet that sorts cards by *meaning* instead of by alphabet. When you ask "what's near this pin?", it finds the closest cards fast, without checking every single one.

Real-world example: a SQL table stores rows and finds them with `WHERE`. A vector database stores chunks and finds them by *similarity*. You'll still use a `WHERE`-style filter on metadata (`topic = 'databases'`), which makes it feel familiar if you know SQL.

1. **Open a persistent client** (`chromadb.PersistentClient`) pointed at a folder such as `chroma_db/`. The default in-memory client forgets everything when your script ends.
2. **Create one collection**, for example `study_notes`. Use one collection with good metadata rather than one collection per topic, so you can search across everything or filter down.
3. **Choose the distance measure.** Set the collection to cosine similarity, which is the standard choice for sentence-transformer embeddings.
4. **Add chunks in batches** with four parallel lists: `ids`, `documents` (the text), `embeddings`, and `metadatas`. Chroma limits how much you can add in one call, so a few hundred per batch is safe.
5. **Keep metadata simple.** Chroma metadata values must be strings, numbers or booleans. Store `topic`, `source_type`, `source_file`, `section`, and `page` (for books and OCR), not nested lists.
6. **Make re-runs safe.** Use `upsert` instead of `add`, or delete and recreate the collection, so running `index.py` twice doesn't double every chunk.
7. **Check it.** `collection.count()` should match the chunk count from Step 2, and `collection.get()` on a few IDs should show the right text and metadata.

Watch out: if you change the chunking rules or the embedding model, rebuild the whole collection. Mixing old and new chunks makes results inconsistent.

Step 5 — Retrieve the right chunks

Goal: for any question, get back the **5 or so most relevant chunks**, preferring your own notes over the big books.

Explain it like I'm 10: You drop a pin for your question on the meaning map and grab the 5 closest cards. If you only want cards from your blue notebook (your own notes), you only look at blue cards.

1. **Embed the question** with the same model from Step 3.
2. **Query the collection** for the top *k* results (`n_results`). Start with `k = 5` . Too few and you miss context; too many and the prompt fills with loosely related text.
3. **Read the distances.** Chroma returns a distance for each hit (smaller = closer). Print them while testing. If even the best hit is far away, your notes probably don't cover the question.
4. **Filter with metadata when it helps.** Chroma's `where` argument filters before ranking, for example `{"topic": "system-design"}` or `{"source_type": "my-notes"}` . Later you could add a `--topic` option to your script.
5. **Stop the books from crowding out your notes.** The two biggest books together are larger than all your notes, so they win many searches by sheer volume. Two simple fixes:
   - **Two searches, then merge:** get the top 3 from `my-notes` + `interview-questions` , and the top 2 from `book` + `ocr` .
   - **Re-score:** fetch the top 15, give `my-notes` hits a small bonus, then keep the best 5.
6. **Remove near-duplicates.** Overlapping windows from Step 2 can return two chunks with mostly the same text. Keep one of them.

Real-world example: ask "What is consistent hashing?" Without a fix, all five hits may come from the 1,354-page topmate System Design PDF. With the two-search merge, you get your own `SYSTEM-DESIGN-NOTES` explanation first and the book as backup.

Next level (optional): *hybrid search* combines embedding search with keyword search (BM25). It helps with exact terms embeddings can blur, like `@Transactional` , `HAVING` or an error code.

Step 6 — Generate the answer with Claude

Goal: send Claude the question plus the retrieved chunks, with instructions to answer **only from those chunks** and **cite where each fact came from**.

Explain it like I'm 10: You hand a smart friend 5 index cards and say: "Answer using only these cards, and tell me which card you used. If the cards don't say, just tell me." That last rule stops your friend from making things up.

1. **Label each chunk** before putting it in the prompt, so Claude can cite it. For example: `[1] databases › sql-complete-notes › Joins > LEFT JOIN` followed by the chunk text, or `[3] java-spring › jhu784 notes, page 412`.
2. **Write the prompt** in three parts: the rules, the labeled chunks (wrapped in XML-style tags such as `<context>`), then the question. A starting template:

```
You are a study assistant. Answer the question using ONLY the
numbered excerpts inside <context>. After each fact, cite the
excerpt number like [2]. If the excerpts do not contain the
answer, say "My notes don't cover this" and stop.
Note: excerpts marked source_type=ocr may contain OCR typos.

<context>
[1] ...
[2] ...
</context>

Question: ...
```

3. **Call the Messages API** with the `anthropic` client. A Sonnet model gives strong answers; a Haiku model is cheaper and faster for quick study questions. Set `max_tokens` to around 800 so answers stay focused.
4. **Print the sources yourself** under the answer, from the metadata of the chunks you sent. Don't rely only on Claude's citations; this list is the ground truth of what was retrieved.
5. **Handle the empty case.** If the best retrieval distance from Step 5 is too far, skip the API call and say the notes don't cover it. This saves money and avoids a forced answer.

Real-world example: ask "Explain the difference between `@Component` and `@Bean`." A good answer quotes your Java notes with `[1]`, adds a detail from the Spring Boot book with `[3]`, and lists both files underneath, so you can open them and check.

Watch out: the tighter the rules, the more honest the answers. If Claude starts adding facts that aren't in any excerpt, make the "ONLY" rule stronger and check what Step 5 actually retrieved.

Step 7 — Test and evaluate

Goal: prove with a small question set that the right chunks come back and the answers are correct, then fix what fails.

Explain it like I'm 10: Before trusting your new librarian, you quiz them on books you've already read. If they keep grabbing the wrong book, you fix how the shelves are labeled, not the person answering.

1. **Write 15–20 test questions** you already know the answers to, spread across topics, and for each one note the file that *should* be found. For example:

Question	Should retrieve
What is the difference between WHERE and HAVING?	`databases/sql-interview-questions`
What does `strict` turn on in tsconfig?	`javascript-web/ts-complete-notes` or `daily-concepts`
What is a baseline model?	Not in rag-docs, so the answer should be "not covered"

2. **Check retrieval first, separately from answers.** For each question, is the expected file in the top 5? The share of questions where it is called *hit rate @ 5*. If retrieval is wrong, no prompt can fix the answer.
3. **Then check the answers.** Correct? Cites the right excerpt? Says "not covered" for the trick question instead of inventing something?
4. **Change one thing at a time** and re-run the same set, so you know what helped.

Common failures and fixes:

What you see	Likely cause	Fix
Every answer cites the big books	Books outnumber your notes	Two-search merge or bonus for `my-notes` (Step 5)

What you see	Likely cause	Fix
Right file, but the chunk is half an idea	Chunks cut mid-section	Split on headings first; add overlap (Step 2)
Chunk is on topic but the answer misses the detail	Detail was past the model's token limit	Smaller chunks or the BGE model (Step 3)
Exact terms like `@Transactional` aren't found	Embeddings blur exact keywords	Add hybrid keyword search (Step 5)
Garbled words in answers	Text came from an `ocr` file	Lower its ranking, or fix the worst OCR pages by hand
Confident answer that isn't in your notes	Prompt rules too loose	Strengthen the "ONLY from excerpts" rule (Step 6)

Real-world tip: keep your test questions in a small file and rerun them after every change. That file plus your before/after hit rate is a great story for interviews: it shows you measured the system instead of guessing.

Next upgrades and top resources

Once steps 1–7 work and your hit rate is steady, add one upgrade at a time and re-run your test questions after each:

- **Add your AI class transcripts** as a new `ai-class` source, chunked by slide.
- **Contextual chunks:** have an LLM write one sentence of context for each chunk ("This is from the SQL notes, section on joins…") before embedding it.
- **Hybrid search + re-ranking:** combine keyword (BM25) and embedding search, then re-rank the top 20 with a cross-encoder model.
- **A simple UI:** wrap `ask.py` in Streamlit or Gradio so you can chat with your notes in a browser.

Top resources:

Resource	Why it's worth your time
Chroma docs: Clients	Official guide to the persistent client and collections you use in Step 4. Short and practical, and it shows metadata filtering with `where`.
Sentence Transformers docs	The library behind Step 3. Its semantic search pages explain embeddings, cosine similarity and batching with runnable examples.
Pinecone: Chunking Strategies for LLM Applications	The clearest explanation of chunk size, overlap and structure-aware splitting, which are the Step 2 decisions that matter most for your mixed notes and books.
Anthropic: Introducing Contextual Retrieval	Explains why chunks lose meaning when cut from their document, and how adding context plus hybrid search cuts failed retrievals. It's the natural next upgrade after this build.
LlamaIndex: Contextual Retrieval cookbook	A worked implementation of the Anthropic technique, useful when you're ready to try that upgrade.